EventBrain AI: Intelligent Event-Saver

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

Team Rolex

Email: umhackathon@um.edu.my

1. Introduction:

The document explains the strategic solution of the identified solution, "EventBrain AI: Intelligent Event-Saver" where the problem statement is highlighted in Product Requirements Documentation.

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decision behind the development of "EventBrain AI: Intelligent Event-Saver"

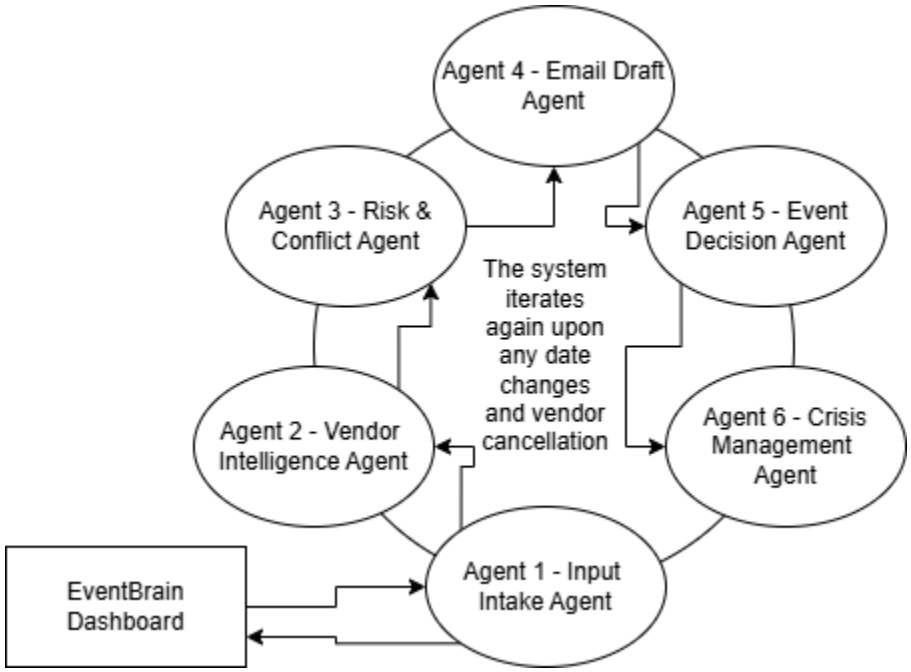
So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *EventBrain AI is an agentic system with a single website panel which features a React frontend for the chat interface and dashboard, a FastAPI Python backend integrating Gemini API for agent reasoning, Socket.io for real-time updates, and Supabase for storing event states, vendor data, and histories. The services are separated into agents where each agent is created to automate each workflow such as Intake Agent, Vendor Intelligence Agent, Risk and Conflict Agent (runs in parallel), Email Drafter Agent, Decision Agent and Crisis Management Agent.*

2. ***Data Flows:***
    a. *The user inputs "Prepare team building activity with 80pax. The budget is RM 4K and the event will be on next Friday". The input is flowed to Intake agent for parsing, then to Vendor Intelligent Agent and Risk & Conflict Agent parallelly to rank the vendors based on budget fit, rating and suitability. In parallel, the risk and conflict agent checks the date if it is a peak season or public holiday and returns a risk score out of 10, specific risk factors and suggest mitigations. Next, it will move to Email Drafter Agent to write one professional inquiry email per vendor, each mentioning the specific date, pax count, and that vendor's speciality. After that, the data flows to Decision Agent to synthesise the previous information received and produces three concrete execution paths: conservative (stay within budget, reduce scope), balanced (slight budget increase, best value), aggressive (full scope, higher cost) and ending its final destination to Crisis Management Agent where the replacement plans are created upon date and vendor cancellation.*

3. ***Model Process:***
    a. Gemini-powered agents process iteratively: Intake extracts params/flags risks, Vendor suggestion and drafts emails, risk scores mitigations, Decision outputs paths, auto draft replacements on crisis like vendor and date cancellations.

    b. *Below is the iterative model of EventBrain AI*



4. *Role of Reference:*
    a. *This document is a reference for all the developers and testers to enable the overall high-level architecture of this project.*

1.2. ***Background***:

Malaysian companies organize 30-50 internal and external events yearly like team buildings, festival celebrations, awards ceremony and corporate workshops. Due to extensive initiatives, many companies failed to take an efficient decision on handling vendor negotiation briefings, budget conflict resolution and real-time crisis decisions, leading to lost oversight, 20-30 hour delays per event, double-bookings, vendor mismanagement and productivity drops from poor time allocation. These manual workflows can't handle unstructured requests ("Plan a 50-person team building next week, budget 5k"), real-time changes (weather issues), or high volumes, resulting in errors, burnout, and missed ROI opportunities.

Aspect	Current Solutions	EventFlow AI
Input Handling	*Manual entry via Google Forms*	*The Gemini parses unstructured chat (can be Malay or English)*
Vendor Management	*Random phone/email hunts*	*Auto-ranks and drafts emails from mock database memory*
Risks Check	*Forgotten (public holidays, festival seasons and fasting periods)*	*Proactive scores/mitigations*
Crises	*Need to plan additional round table discussions at the last minute.*	*Iterative re-run and auto-alerts Memory*

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Company Event coordinator	• *Input the event's basic details (Date, budget, event type and number of pax)* • *Approve event decisions according to their convenient* • *Handle the crisis upon received date and vendor cancellation.*	• *Receives top suitable vendors for the event.* • *Receives event execution paths: conservative (stay within budget, reduce scope), balanced (slight budget increase, best value), aggressive (full scope, higher cost).* • *Live iteration for mitigation plans if received date or vendor cancellation.*
Human Resources Department		
Vendors	• *Input their details and things which they provide for an event with the budget*	• *Wide range exposure from various companies*

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

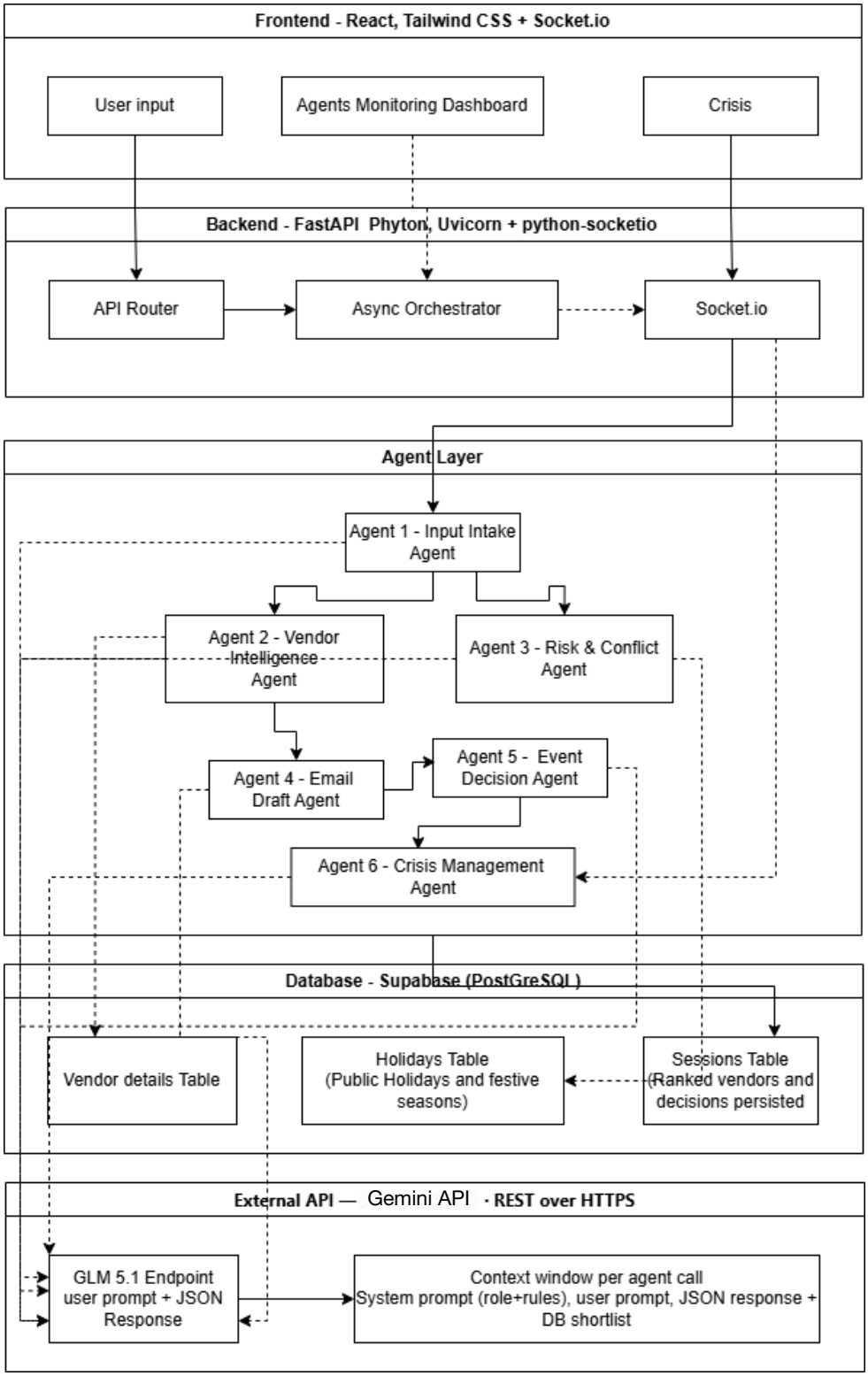
Type	Details
System	*Web Application*
Architecture	• *Frontend - React 18 (Vite) , Tailwind CSS , Socket.IO (real-time updates)* • *Backend - FastAPI (Python), Uvicorn + python-socketio (real-time)* • *AI & Processing - Gemini API (LLM via REST) and Async orchestration (parallel agents)* • *Database - Supabase (PostgreSQL)*

EventBrain AI features an event-driven, real-time multi-agent architecture where React 18 (Vite + Tailwind + Socket.IO) provides a single-panel chat interface for coordinators to input requests like "80pax team building RM4k next Friday," instantly streaming live agent iterations via Socket.IO to FastAPI (Python + Uvicorn + python-socketio) backend that orchestrates five parallel Gemini API agents—Intake (parsing/risk flags), Vendor Intelligence, Risk & Conflict (Malaysia holidays/weather), Email Drafting for vendors, Decision (3 execution paths), and Crisis Management—while Supabase PostgreSQL stores event state with real-time subscriptions that trigger workflow re-runs on vendor cancellations, delivering sub-2s end-to-end latency through async processing and structured JSON reasoning for seamless corporate event management.

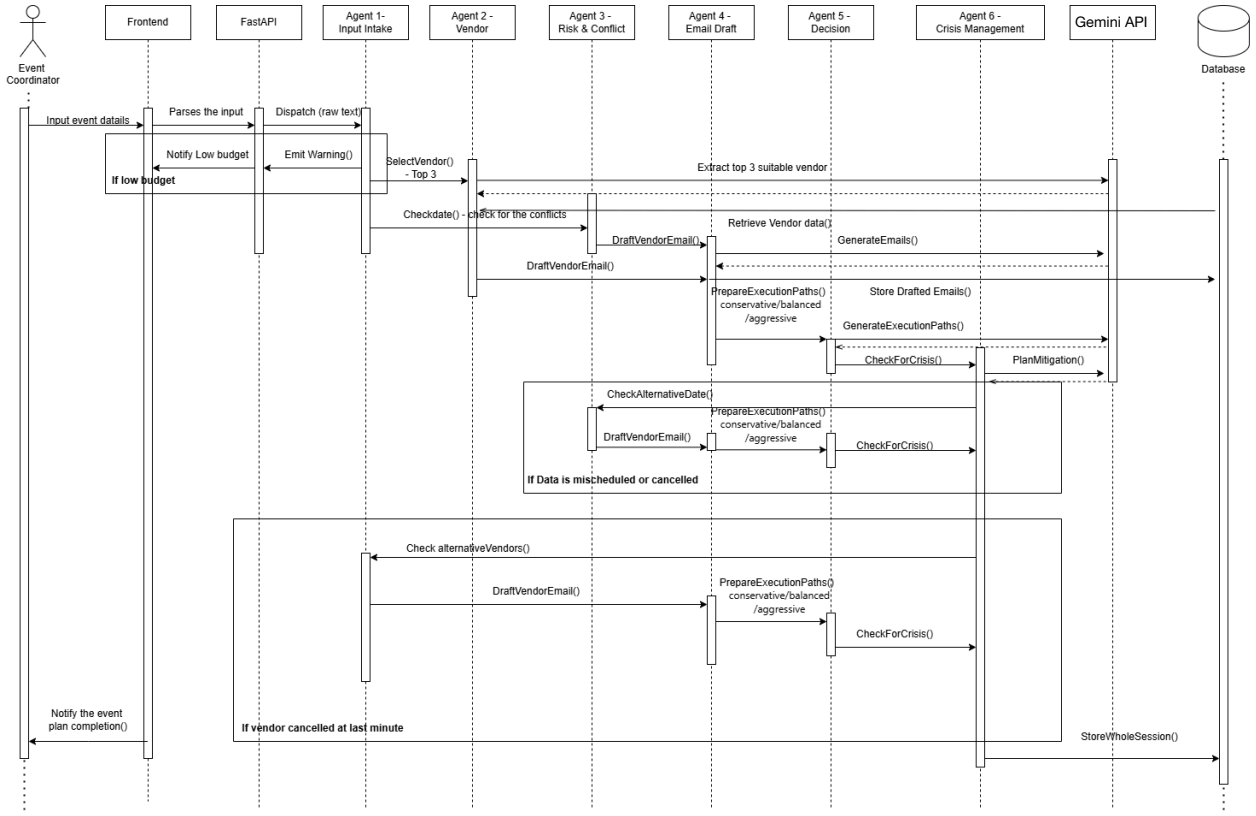
2.1.2 LLM as Service Layer

EventFlow AI integrates **Gemini API as its central service layer** through FastAPI's async orchestrator making five parallel REST API calls, not as a generic AI module, where each agent (Intake, Vendor Intelligence, Risk & Conflict, Decision, Notification) consumes structured JSON prompts with tool-calling schemas to produce standardized outputs like vendor_rank[], risk_score, and decision_paths[], streamed live (stream: true) via Socket.IO to React frontend while Supabase PostgreSQL persists event state and triggers crisis re-runs through real-time subscriptions, ensuring sub-2s iterative reasoning for corporate event management with comprehensive mock fallbacks for Gemini API outages. The following Dependency diagram includes:

2.1.3 Dependency Diagram



2.2.2. Sequence Diagram



2.2. Technological Stack

2.2.1. Frontend [React Vite + Tailwind CSS with Socket.io (real time updates)]

2.2.2. Backend [Python FastAPI + Uvicorn + Python-socketio (real-time)]

2.2.3. AI & Processing Engine [Gemini API (LLM via REST API)]

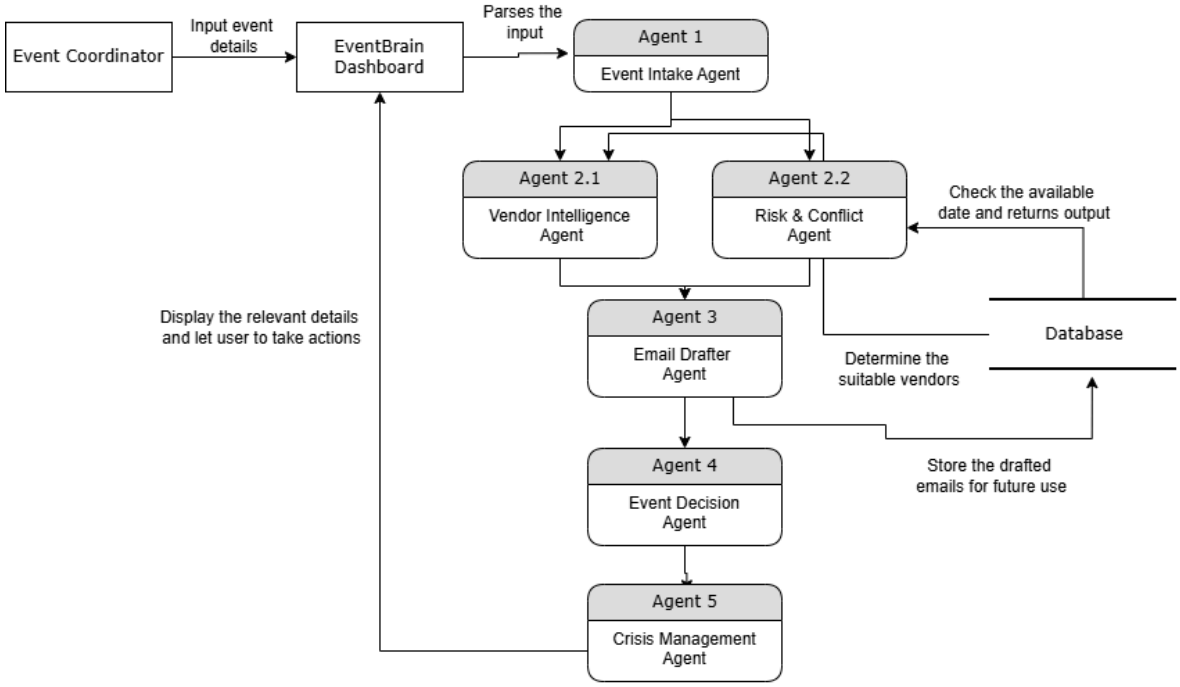
2.2.4. Database [Supabase - PostgreSQL – Relational Database for vendors, crisis, events and mitigation plans]

Email: umhackathon@um.edu.my

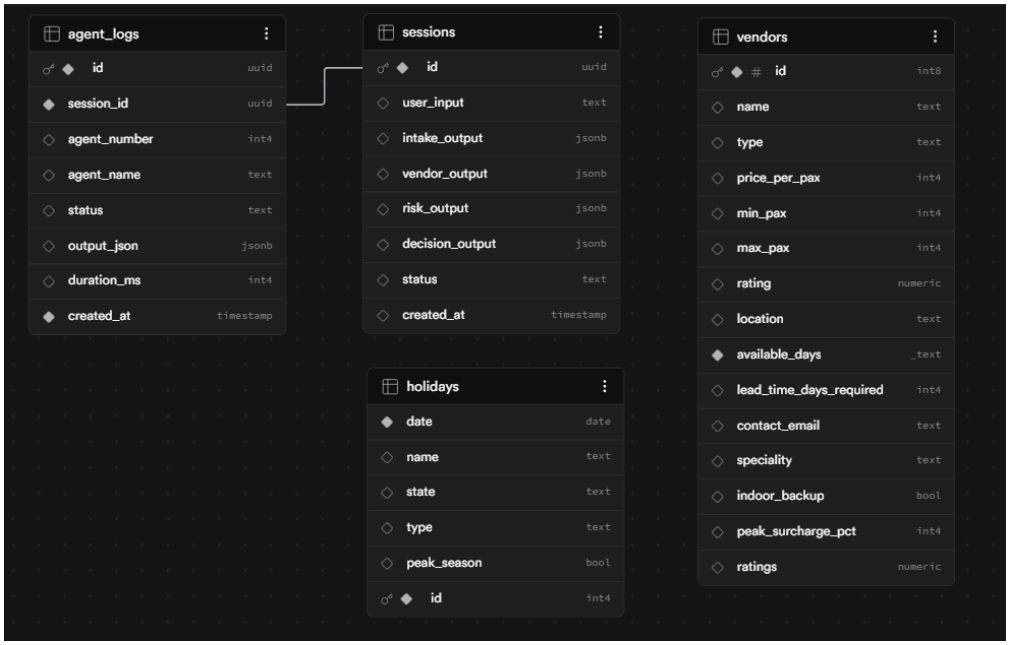
2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema (Normalized 3NF - ERD)



3. Functional Requirements & Scope

This section highlights the core boundaries of EventBrain AI, focussing on its technical feasibility.

3.1. Minimum Viable Product:

3.1.1. *Below are the EventBrain's core features which will be showcased by our demonstration.*

No.	Features	Description
1	*Event details intake agent*	Extracts structured data, pax count, budget, date, event type, location and calculates budget per pax, immediately flags if it's too low for the market rate. The agent returns a clean JSON object so that every other agent can read easily.
2	*Vendor ranking*	The agent runs parallel with Risk & Conflict agent and queries the database in Python to filter vendors that physically fit the right pax range, enough lead time and the correct event type. Then, it shortlist to Gemini and asks it to rank the top 3 intelligently based on budget fit, rating, and suitability.
3	*Risk and conflict detection*	The agent runs parallel with Vendor Intelligence Agent, queries the holidays table to check if the requested date is a public holiday or peak season. The agent passes the holiday result, weather context for KL in that month, budget risk, lead time to Gemini and asks it to reason about combined risk. This also returns a risk score out of 10, specific risk factors, and suggested mitigations.
4	*Drafts vendor emails*	The Email Agent asks Gemini to write one professional inquiry email per vendor, each mentioning the specific date, pax count, and that vendor's speciality.
5	*Manages last-minute crisis*	Once the crisis button is clicked, the Crisis Management Agent removes the cancelled vendor in Python first, then re-queries the database for all remaining vendors and passes them to Gemini. This produces three replacement vendor options with urgent pre-written emails and a revised execution plan

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Performance	*<2s end-to-end agent iteration latency; handle 50+ concurrent events without degradation*	*FastAPI async endpoints with Uvicorn, Socket.IO real-time streaming and Supabase real-time subscriptions*
Reliability	*99% agent success rate and handle Gemini API downtime*	*Async retry logic (3x exponential backoff), Supabase transactions for event state*
Maintainability	*<1 day for new agent deployment; self-documenting APIs*	*FastAPI OpenAPI auto-docs, modular agent classes in Python and Supabase SQL editor*
Observability	*Real-time monitoring dashboard*	*Supabase logs + analytics, Socket.IO event tracking and agent execution tracing*
Token Latency	*<2s end-to-end agent iteration latency, <500ms token latency for Gemini responses and handle at least 10+ concurrent events* ***(Based on the token received by UMH2026)***	*FastAPI async endpoints + Uvicorn; Socket.IO streaming; Supabase real-time subscriptions; Gemini prompt compression + parallel agent calls*
Cost Efficiency	*<RM 0.20 per event workflow; 50-80% LLM token cost reduction*	*Semantic caching for repeated vendor/event queries; structured JSON outputs from Gemini; batch small agent calls; Supabase Edge Functions for lightweight tasks*

3.3. Out of Scope / Future Enhancements

Below are the high-level features that the team planned but not part of this UMHackathon 2026.

a. *User authentication module - The login and signup page for vendors and event coordinators.*
b. *Vendor registration module - The page where vendor promote themselves and the companies can get direct information rather than web scrapping throughout Google and Safari*
c. *Crisis Management automation - The crisis will only be handled once user clicks on the crisis button*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

EventFlow AI implements progressive delivery for pre-deployment validation. New agent features (enhanced Risk Agent for Malaysia festive seasons) initially tested with 5% simulated event load via Supabase feature flags. Socket.IO metrics track agent latency (<2s), Gemini token costs (<RM 0.20/event-estimation only) and approval rates. Once validated locally, traffic scales to a 100% staging environment. Post-deployment, A/B testing compares agent variants (conservative vs. aggressive decision paths) measuring vendor response time and budget adherence.

4.1.2. Emergency Rollback (ER) & Golden Release:

Golden Release (v0.9-prelim) maintained as Docker image in GitHub Container Registry. ER triggers automatically if P1 incidents detected during development:

- *Agent iteration latency >5s (99th percentile)*
- *Gemini token cost <RM0.20/event (estimation only)*
- *Supabase connection pool exhaustion*
- *GitHub Actions CI/CD pipeline reverts to Golden Release within 90 seconds preserving Socket.IO connections while logging to Supabase analytics.*

4.1.3. Priority Matrix:

4.1.3.1.

Priority	Trigger Condition	Action Taken	Response Time
P1 Critical	*Agent latency >5s; Supabase 5xx errors >1%; Security violations*	*Emergency Rollback to Golden Release v0.9*	*<90 seconds*
P2 High	*Vendor ranking accuracy <90%; Gemini rate limits hit*	*Auto-scale Uvicorn dev workers; Prompt cache refresh*	*<10 minutes*
P3 Medium	*UI responsiveness <3s; Socket.IO disconnect spikes*	*Vite dev server restart; Connection retry logic*	*<30 minutes*
P4 Low	*Tailwind styling issues; i18n text glitches*	*Next development sprint*	*Within a day*

4.2. Monitoring:

4.2.1. Development Health Dashboard (React + Tailwind)

- *Agent Latency: 1.1s avg (P1 alert if >5s)*
- *Token Usage: 2.2k/event ($RM0.10)*
- *Active Events: 12/50 dev capacity*
- *Error Rate: 0.1% (P1 if >1%)*
- *Supabase Dev Connections: 8/100*

Alert Triggers

- Supabase Studio: Monitor PostgreSQL deadlocks, connection limits
- Uvicorn Access Logs: Track FastAPI endpoint latency, 5xx errors
- Gemini Dashboard: Token consumption spikes, rate limit warnings
- GitHub Notifications: CI/CD pipeline failures, Docker build issues

Damage Detection: 1-minute rolling window anomaly detection flags P1 issues, local circuit breakers pause agent iterations during Gemini dev quota exhaustion.

4.3. Assumptions:

a. *Users have stable internet connection (>10Mbps) for Socket.IO + Supabase streaming*
b. *Local vendor database (CSV/JSON) available for mock data during preliminary round*
c. *Event coordinators provide clear budget authority (<RM5k) in test scenarios*
d. *2026 Malaysia public holidays calendar (Awal Ramadhan ~Feb 19) for Risk Agent validation*
e. *Gemini dev tier maintains <500ms token latency during demo hours*

4.4. External Dependencies:

Tools	***Purpose***	***Risks***
Gemini API	*Core LLM reasoning engine handles prompt inference, multi-step agent reasoning and structured JSON outputs (Intake, vendor rankings, risk scores, email drafts, event decision paths and crisis management)*	*High - Rate limits or downtime breaks all 5 agents, The demo failure during preliminary round*
Supabase PostgreSQL	*Event state persistence, real-time subscriptions, Row-Level Security for multi-user access*	*Medium - Connection pool exhaustion during load spikes; dev project limits*

Socket.IO (python-socketio + client)	*Real-time streaming of agent iterations, live crisis alerts, dashboard updates*	*Medium - Network partitions disconnect coordinators mid-workflow*

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

Activity Plan	16/4	17/4	18/4	19/4	20/4	21/4	22/4	23/4	24/4	25/4	26/4
Domain Reveal											
Idea Planning											
Preparation of Project Requirement Document											
Frontend											
Backend - Agent 1 to Agent 3 with Gemini											
Backend - Agent 4 to Agent 6 with Gemini											
Database Preparation											
Integration thorugh socket.io											
Preparation of SAD and QA Testing Document											
Demo Video & Submission Prep											

Email: umhackathon@um.edu.my

5.2. ***Team Members Role:***

Name	Role
Kishant A/L R.Kunasegaran	• Project Manager • Backend Setup with AI integration for - Agent 4 - Draft Email Agent - Agent 5 - Event Decision Agent - Agent 6 - Crisis Management Agent
Haresh A/L Anna Ravi Maran	• Database setup for EventBrain AI - Manages mock vendor data - Setup connection to backend • Preparation of Product Requirement Documentation
Sanjeevan A/L Kumaresan	• Facilitates the integration between frontend and backend • Preparation QA Testing Document • Plan test coverage
Ruben Raj A/L Sivaperumal	• Fully functional frontend Setup • Presentation Deck preparation • 10-min presentation video plan
Jayasharma Naidu A/L N.D.Narendran	• Backend Setup with AI integration for - Agent 1 - Input Intake Agent - Agent 2 - Vendor Intelligence Agent - Agent 3 - Risk & Conflict Agent • Preparation of System Analysis Documentation

5.3. *Recommendations:*

Area	Recommendation	Impact
Database	*Enable Supabase connection pooling and read replicas for agent results queries*	*Prevent connection exhaustion during peak loads*
Caching	*Redis for vendor rankings and frequent risk data (holidays/weather)*	*80% cache hit rate and sub-500ms responses*
Context Delivery Network + Edge	*Vercel Edge for React and Cloudflare for static assets*	*Global latency <150ms*

Email: umhackathon@um.edu.my